

JWT, Cookies, Sessions & Bcrypt Lecture Notes

1. What Are They and Why Do We Need Them?

The Core Problem: HTTP is Stateless

Meaning: The HTTP protocol (which powers the web) is "stateless." This means that every time a client (browser) sends a request to the server, the server treats it as a brand-new, completely independent request. The server has NO memory of past requests.

Why we need them: If HTTP has no memory, how does a website keep you logged in when you move from the Home Page to your Profile Page? The answer is state management tools: Sessions, Cookies, and JWTs. They allow us to "remember" the user across multiple requests.

- Sessions: Stored on the Server. The server creates a unique session ID for the user and stores the user's data in its own memory/database. It sends only the ID to the browser.
- Cookies: Stored in the Browser. They are small text files that the browser automatically sends back to the server with every request.
- JWT (JSON Web Tokens): Stored on the Client (often in a cookie or local storage). It is a self-contained token that holds the user's data securely without the server needing to save a session in its database.

2. What is a JWT?

Meaning: JWT stands for JSON Web Token. It is an open standard used to share security information between two parties — a client and a server. It is primarily used for Authentication (logging users in) and Authorization (checking if they have permission to access certain data).

Because a JWT is cryptographically signed, the server can trust the data inside it without needing to look it up in a database every time.

3. The Structure of a JWT

A JWT is simply a long string of text divided into THREE parts, separated by dots (.).

Format: xxxxx.yyyyy.zzzzz

1. HEADER (Algorithm & Token Type)	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
------------------------------------	--------------------------------------

2. PAYLOAD (The actual Data / Claims)	eyJzdWliOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIn0
3. SIGNATURE (To verify authenticity)	SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

What does each part do?

1. Header: Contains metadata about the token. Specifically, it tells the server what type of token it is (JWT) and what algorithm was used to encrypt/sign it (e.g., HMAC SHA256 or RSA).
2. Payload: Contains the 'claims' or the actual data. This is where you put the user's ID, username, and role (e.g., admin). Note: The payload is encoded, NOT encrypted! Do not put passwords or sensitive data here, as anyone can decode and read it.
3. Signature: The most important part. The server takes the encoded Header, the encoded Payload, and a Secret Key (that only the server knows) and hashes them together. If a hacker tries to change the payload, the signature will become invalid, and the server will reject the token.

4. JWT in Express: Creating and Checking Tokens

To use JWT in Node.js, we install the package: `npm install jsonwebtoken`

Code:

```
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());

const SECRET_KEY = 'my_super_secret_key'; // In real apps,
store this in .env

// API 1: Create Token (Login)
app.post('/api/login', (req, res) => {
  // Normally, you check DB for username & password here
  const user = { id: 101, username: 'umar_dev', role:
'admin' };

  // Create the JWT
  // jwt.sign(payload, secret_key, options)
  const token = jwt.sign(user, SECRET_KEY, { expiresIn: '1h'
});

  res.status(200).json({
    message: 'Login successful',
```

```

        token: token
    });
});

// API 2: Check Token (Protected Route)
app.get('/api/profile', (req, res) => {
    // The client sends the token in the headers
    (Authorization: Bearer <token>)
    const authHeader = req.headers['authorization'];

    if (!authHeader) {
        return res.status(401).json({ message: 'Access Denied.
No token provided.' });
    }

    // Extract token from "Bearer <token>"
    const token = authHeader.split(' ')[1];

    // Verify the token
    jwt.verify(token, SECRET_KEY, (err, decodedData) => {
        if (err) {
            return res.status(403).json({ message: 'Invalid or
Expired Token' });
        }
        // Token is valid! decodedData contains the payload
        (id, username, role)
        res.status(200).json({
            message: 'Welcome to your profile!',
            userData: decodedData
        });
    });
});

app.listen(3000, () => console.log('Server running on port
3000'));

```

5. Cookies and Their Use

Meaning: A cookie is a small piece of data that a server sends to the user's web browser. The browser stores it and automatically sends it back to the server with every subsequent request to that same domain.

Use Cases: Cookies are heavily used for Session Management (keeping users logged in), Personalization (dark mode preferences), and Tracking (Google Analytics).

Why combine JWT and Cookies? Instead of forcing the frontend developer to manually attach the JWT to every single API request header, we can place the JWT inside an HTTP-only Cookie. The browser will then securely and automatically send the JWT back to the server for us!

6. Password Security: Hashing, Salt, and Bcrypt

Saving plain-text passwords in a database is highly insecure. If a database is hacked, the attacker gets everyone's passwords. We solve this using Hashing and Salting.

What is Hashing?

Meaning: Hashing is a one-way mathematical function that scrambles data (like a password) into a fixed-length string of characters. Crucially, it is ONE-WAY: you cannot decrypt or reverse a hash back into the original password. You can only verify it by hashing the user's input again and comparing the two hashes.

What is Salt?

Meaning: A "salt" is random data added to a password BEFORE it is hashed.

Why we need it: If two users have the same password (e.g., "password123"), their hashes would normally look exactly the same. Hackers use "Rainbow Tables" (lists of pre-computed hashes) to crack these. By adding a unique, random salt to every user's password, we ensure that even identical passwords result in completely different hashes.

What is Bcrypt?

Meaning: Bcrypt is a popular, industry-standard NPM package specifically designed for hashing passwords securely. It automatically handles salting and is deliberately designed to be "slow" to compute, which protects against brute-force attacks.

Installation: ``npm install bcrypt``

Basic Bcrypt Example

Code:

```
const bcrypt = require('bcrypt');

async function testHashing() {
  const plainTextPassword = 'mySuperSecretPassword123';

  // 1. Hash the password (10 is the "salt rounds" - higher
  // is slower but more secure)
  const hashedPassword = await
  bcrypt.hash(plainTextPassword, 10);
  console.log('Hashed:', hashedPassword);

  // 2. Compare a login attempt to the hashed password
```

```

    const isMatch = await
bcrypt.compare('mySuperSecretPassword123', hashedPassword);
    console.log('Do passwords match?', isMatch); // true
}
testHashing();

```

7. Full Example: Signup, Login, and Dashboard using MongoDB, Cookies, and Bcrypt

For this to work, you need to install: `npm install express mongoose jsonwebtoken cookie-parser bcrypt`

```

Code:
const express = require('express');
const mongoose = require('mongoose');
const jwt = require('jsonwebtoken');
const cookieParser = require('cookie-parser');
const bcrypt = require('bcrypt');

const app = express();

// Middlewares
app.use(express.json()); // Parses JSON bodies
app.use(cookieParser()); // Parses cookies from incoming requests

const SECRET = 'super_secret_jwt_key';

// --- 1. MongoDB Connection & Schema ---
mongoose.connect('mongodb://localhost:27017/AuthApp')
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.log(err));

const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true }
});
const User = mongoose.model('User', userSchema);

// --- 2. SIGNUP Route (Using Bcrypt to Hash Password) ---
app.post('/signup', async (req, res) => {
  try {
    const { username, password } = req.body;

    // Hash the password with bcrypt (10 salt rounds)

```

```

    const hashedPassword = await bcrypt.hash(password,
10);

    // Create user in MongoDB with the HASHED password
    const newUser = await User.create({
        username: username,
        password: hashedPassword
    });

    res.status(201).json({ message: 'User created
successfully', user: newUser });
    } catch (error) {
        res.status(400).json({ message: 'Signup failed', error
    });
    }
});

// --- 3. LOGIN Route (Compare Hash & Store Token in Cookie) -
--
app.post('/login', async (req, res) => {
    try {
        const { username, password } = req.body;

        // Find user in DB
        const user = await User.findOne({ username });
        if (!user) {
            return res.status(401).json({ message: 'Invalid
credentials' });
        }

        // Compare plain-text password from login with the
hashed password in DB
        const isMatch = await bcrypt.compare(password,
user.password);
        if (!isMatch) {
            return res.status(401).json({ message: 'Invalid
credentials' });
        }

        // Generate JWT Token
        const token = jwt.sign({ id: user._id, username:
user.username }, SECRET, { expiresIn: '1h' });

        // Set the token inside a Cookie using maxAge (1 hour
in milliseconds)

```

```

        res.cookie('authToken', token, {
            maxAge: 3600000, // 1 Hour (1000ms * 60s * 60m)
            httpOnly: true, // Prevents cross-site scripting
                                (XSS) attacks
        });

        res.status(200).json({ message: 'Login successful!
Cookie set.' });
    } catch (error) {
        res.status(500).json({ message: 'Login error', error
    });
    }
});

// --- 4. DASHBOARD Route (Checks Cookie) ---
app.get('/dashboard', async (req, res) => {
    try {
        // Read the token from the cookies (made possible by
        cookie-parser)
        const token = req.cookies.authToken;

        if (!token) {
            return res.status(401).json({ message: 'Access
Denied. Please login first.' });
        }

        // Verify the token
        const decodedData = jwt.verify(token, SECRET);

        // Check if user still exists in DB using the ID from
        the token
        const user = await User.findById(decodedData.id);
        if (!user) {
            return res.status(404).json({ message: 'User no
longer exists in DB.' });
        }

        res.status(200).json({
            message: 'Welcome to the Dashboard!',
            user: {
                id: user._id,
                username: user.username
            }
        });
    } catch (error) {

```

```
        res.status(403).json({ message: 'Invalid or Expired  
Token. Please login again.' });  
    }  
});  
  
// --- Start Server ---  
app.listen(3000, () => console.log('Server running on port  
3000'));
```